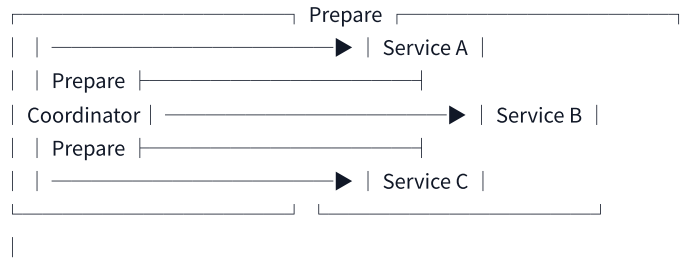
TOPIC 1: DISTRIBUTED TRANSACTIONS — BEYOND SAGA

Two-Phase Commit (2PC) — Why Microservices Avoid It

text



Aspect	2PC	Saga
Locking	Yes (blocks resources)	No
Consistency	Strong	Eventual
Availability	Lower	Higher
Scalability	Poor	Excellent
Microservices fit	❌ No	✅ Yes

Interview Insight: "2PC doesn't work in microservices because locks can't span services and network partitions kill availability."

XA Transactions vs Saga vs TCC (Try-Confirm-Cancel)

Pattern	Locking	Compensating	Best For
XA / 2PC	Long locks	Rollback	Monoliths, not microservices
Saga	No locks	Compensating actions	Long-running workflows
TCC	Short "try" locks	Explicit cancel	High-performance scenarios

TCC Pattern Explained

text

Phase 1: TRY
- ├── Reserve resources (soft lock)
- ├── No actual business execution
- └── Return confirmation token

Phase 2: CONFIRM (if all TRY succeed)
- ├── Execute actual business
- └── Release soft locks

Phase 2 (alt): CANCEL (if any TRY fails)
- ├── Rollback reservations
- └── Release soft locks

When TCC > Saga: Financial transactions where double-spending must be prevented during the reservation phase.

TOPIC 2: EVENT DRIVEN ARCHITECTURE — DEEP DIVE

Event Sourcing vs Change Data Capture vs Event Streaming

text

EVENT PATTERNS COMPARED

Event Sourcing: State = Replay(All Events)
["added", "changed", "deleted"]
Source of truth = Event Store

Change Data Capture: Monitor DB Log → Publish Events
Debezium → Kafka
Non-invasive, existing DB stays source

Event Streaming: Kafka/Kinesis/Pulsar
Infinite replay window
Real-time + batch in one

```
└──────────────────────────────────────────────────────────────────────────┘
```

When to Use Which

Pattern	When to Choose
Event Sourcing	Need full audit trail, time travel, debugging past states
CDC	Existing monolith, can't change application code
Event Streaming	Real-time analytics, data pipelines, Kafka as backbone

Idempotent Event Processing — Deep Dive

text

```
┌──────────────────────────────────────────────────────────────────────────┐
│ IDEMPOTENT EVENT PROCESSOR PATTERN │
├──────────────────────────────────────────────────────────────────────────┤
│ │
│ Consumer: │
│ 1. Receive event { id: "evt-123", type: "order.created" } │
│ 2. Check Redis SETNX "processed:evt-123" │
│ 3. If redis says "already seen" → SKIP │
│ 4. If new → process + store result in DB │
│ 5. Atomic commit: DB update + Redis mark │
│ │
│ Exactly-Once Semantics = Idempotent Consumer + Transactional │
│ Outbox + Kafka idempotent producer │
│ │
└──────────────────────────────────────────────────────────────────────────┘
```

TOPIC 3: API GATEWAY — ADVANCED PATTERNS

Gateway Responsibilities — Complete Map

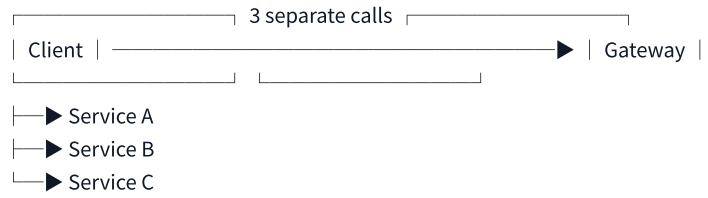
text

```
┌──────────────────────────────────────────────────────────────────────────┐
│ API GATEWAY RESPONSIBILITIES │
├──────────────────────────────────────────────────────────────────────────┤
│ │
│ Layer 1: Security │
│ ├─── JWT/OAuth2 validation │
│ ├─── API key management │
│ ├─── IP whitelisting │
│ └─── Bot detection │
│ │
│ Layer 2: Traffic Control │
│ ├─── Rate limiting (per user/per endpoint) │
│ ├─── Circuit breaking │
│ ├─── Retry with backoff │
│ └─── Request deduplication │
│ │
│ Layer 3: Transformation │
│ ├─── Request/response transformation │
│ ├─── GraphQL → REST aggregation │
│ ├─── Protocol translation (gRPC → HTTP) │
│ └─── Response caching │
│ │
│ Layer 4: Observability │
│ ├─── Distributed tracing header injection │
│ ├─── Request/response logging │
│ ├─── Metrics collection │
│ └─── Alerting │
│ │
└──────────────────────────────────────────────────────────────────────────┘
```

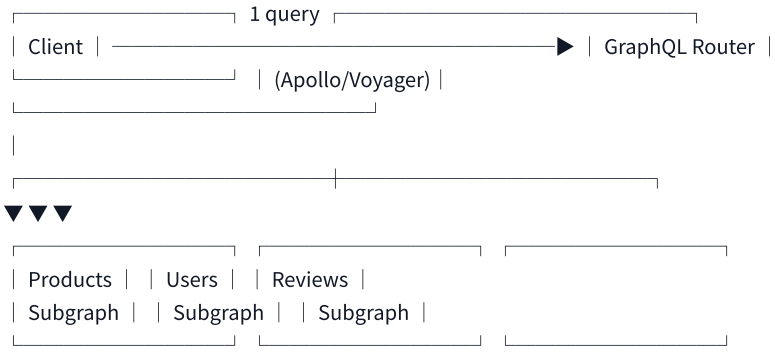
Gateway Implementation Comparison

Gateway	Best For	Notable Features
Kong	Enterprise	Plugin ecosystem, DB-backed
NGINX	High performance	Low latency, small memory
Envoy	Service mesh integration	xDS protocol, gRPC native

Spring Cloud Gateway Java ecosystem Reactive, Spring integrated
AWS API Gateway Serverless Lambda integration, managed
GraphQL Federation vs API Gateway Aggregation
text
API Gateway Aggregation:

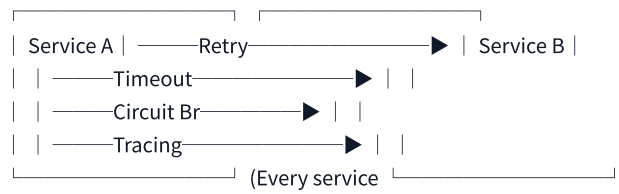


GraphQL Federation:



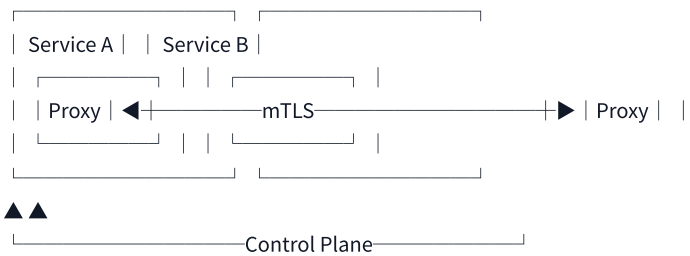
Interview Insight: "API Gateway aggregates at request level; GraphQL federation aggregates at field level with type safety."

TOPIC 4: SERVICE MESH — ENVOY, ISTIO, LINKERD
What Problem Service Mesh Solves
text
Without Service Mesh:



implements its own!)

With Service Mesh:



(Istio/Pilot, Linkerd control)
Control Plane vs Data Plane
Plane Responsibility Components
Data Plane Handles actual traffic Envoy proxies (sidecars)
Control Plane Manages configuration Pilot, Mixer, Citadel (Istio)
Service Mesh Features Matrix
Feature Envoy Istio (on Envoy) Linkerd
mTLS ✓ ✓ ✓
Retry/Timeout ✓ ✓ ✓
Circuit Breaking ✓ ✓ ✓
Traffic splitting ✓ ✓ ✓
Fault injection Limited ✓ Limited
Complexity Medium High Low
When to Adopt Service Mesh

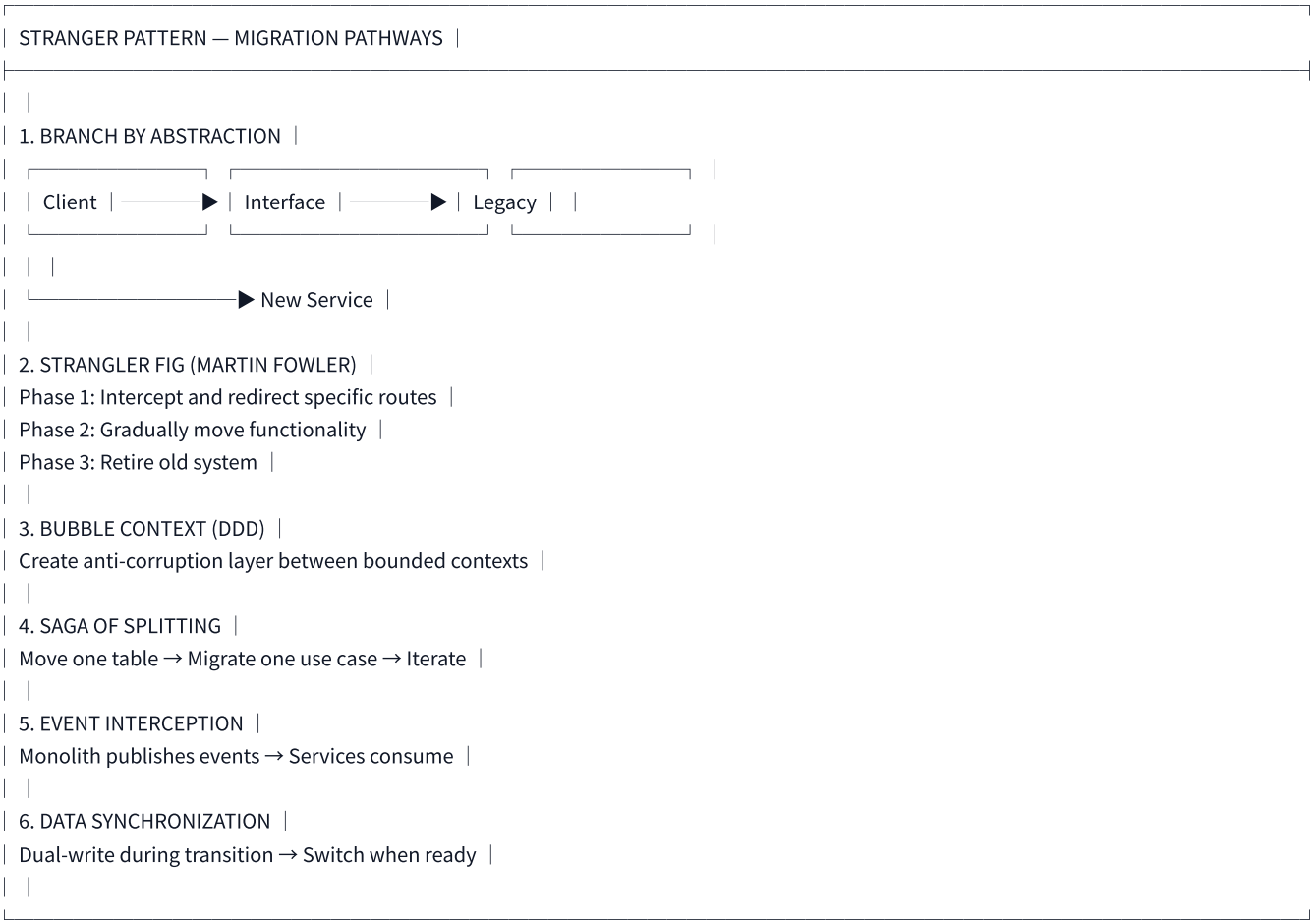
text
✅ GOOD FIT:

- Many services (50+)
- Multiple languages (Java, Go, Python, Node)
- Security/compliance requiring mTLS
- Dedicated platform team exists

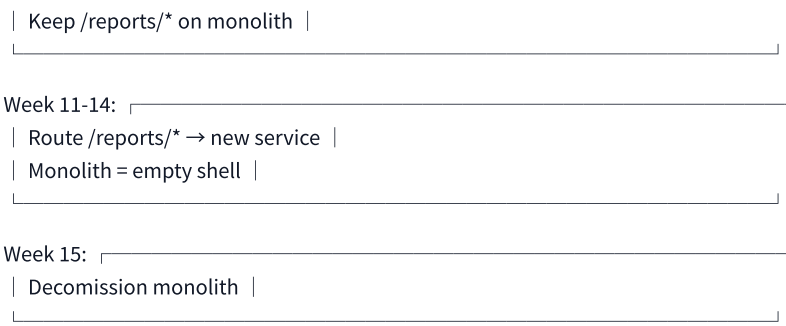
❌ NOT NEEDED:

- < 10 services
- Single language (library approach works)
- No security requirements
- Small team, no platform expertise
  Interview Insight: *"Service mesh is NOT for everyone. For 10-20 services, client libraries (Resilience4j, Hystrix) are simpler. At 50+ services, the standardization of service mesh pays off."*

TOPIC 5: STRANGER PATTERN — MONOLITH TO MICROSERVICES
The Six Stranger Pattern Strategies
text



Strangler Fig — Detailed Timeline
text



Interview Insight: "Never do Big Bang rewrites. The Strangler Pattern is the only proven safe path to microservices."

TOPIC 6: DORA METRICS & MICROSERVICES MATURITY

The Four Key DORA Metrics

Metric	Definition	Target (Elite)
Deployment Frequency	How often deploy to production	Multiple times/day
Lead Time for Changes	Code commit → deployment	< 1 hour
Time to Restore	Incident → resolution	< 1 hour
Change Failure Rate	% of deployments causing incidents	0-15%

Microservices Maturity Model

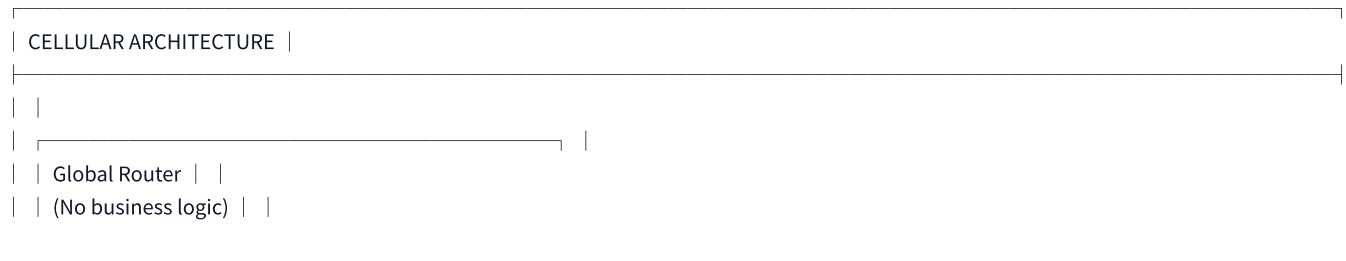
text

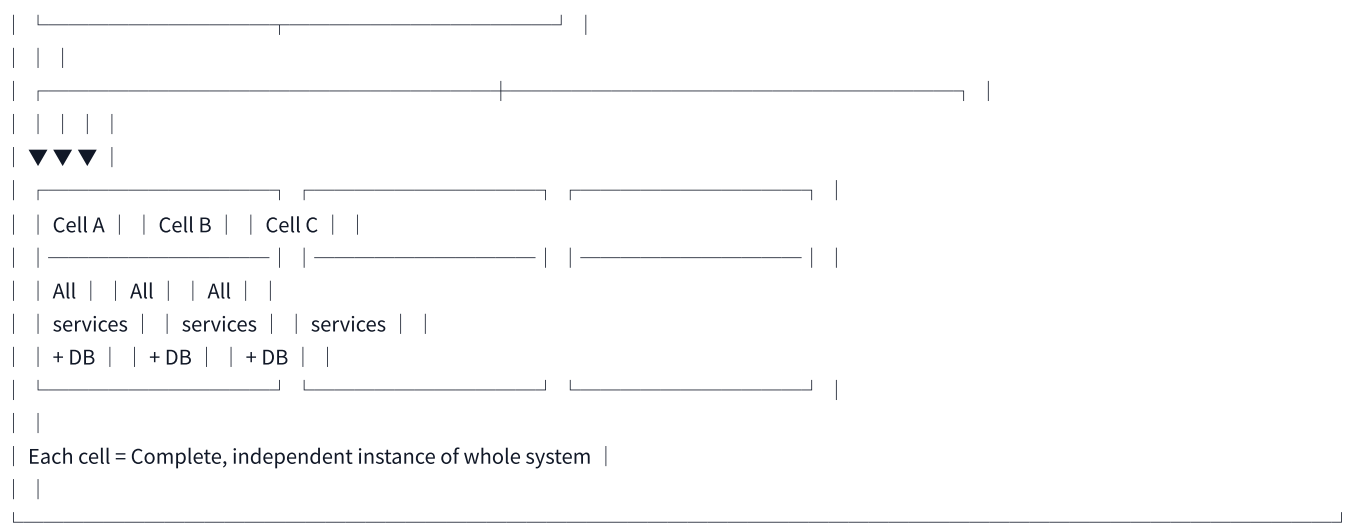
```
Level 1: MONOLITH
 ├── Single deployment unit
 ├── Shared database
 └── Deploy frequency: Monthly

Level 2: MICROSERVICES NAIVE
 ├── Split by technical layers (not domains)
 ├── Shared database hidden
 ├── Synchronous calls everywhere
 └── Deploy frequency: Weekly

Level 3: DOMAIN-DRIVEN
 ├── Split by business domains
 ├── Database per service
 ├── Hybrid sync/async
 ├── Saga for transactions
 └── Deploy frequency: Daily

Level 4: OPERATIONALLY EXCELLENT
 ├── Fully independent deployments
 ├── Blue-green/canary
 ├── Service mesh
 ├── Platform engineering team
 ├── DORA Elite metrics
 └── Deploy frequency: Multiple times/day

Level 5: CELLULAR ARCHITECTURE
 ├── Self-contained cells
 ├── Regional isolation
 ├── Chaos engineering in production
 └── Deploy frequency: Hourly
```

TOPIC 7: CELLULAR ARCHITECTURE — THE NEXT STEP

What is Cellular Architecture

text





Why Cellular > Traditional Microservices

Aspect	Traditional Microservices	Cellular Architecture
Blast radius	Full system	Single cell
Deployment risk	Global	Per cell (canary cells)
Scaling	Per service	Whole cells
Regional failover	Complex	Built-in (cells per region)
Tenancy	Hard	Easy (tenant → cell mapping)
Complexity	High per service	High per cell but isolated

Companies Using Cellular Architecture

Company	Scale	Why Cells
Uber	Thousands of services	Blast radius containment
Netflix	Global streaming	Regional isolation
DoorDash	10K+ microservices	Developer independence

TOPIC 8: DATABASE PER SERVICE — DEEP TRADE-OFFS

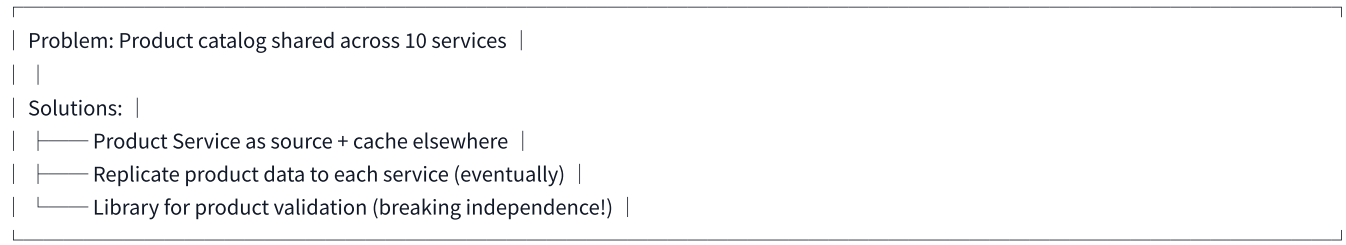
The Hidden Costs

text

Cost 1: Distributed Transactions

Cost 2: Reporting

Cost 3: Shared Reference Data



When Database Per Service Fails

text
❌ Anti-patterns to recognize:

1. Too many join operations across services
   → Suggests wrong service boundaries
2. Report queries constantly aggregating
   → Needs CQRS or data warehouse
3. Shared reference data changing rapidly
   → Cache invalidation becomes nightmare
4. Transactional boundaries span 5+ services
   → Maybe the monolith was better

TOPIC 9: CHAOS ENGINEERING

Principles of Chaos Engineering

text

```
┌──────────────────────────────────────────────
│ CHAOS ENGINEERING MATURITY MODEL │
├──────────────────────────────────────────────
│ │
│ Level 0: No testing │
│ │
│ Level 1: Test in staging only │
│ │
│ Level 2: Test in production with small blast radius │
│ Example: Kill one pod out of 1000 │
│ │
│ Level 3: Production experiments with automated rollback │
│ Example: 2% latency injection, rollback if error rate >0.1% │
│ │
│ Level 4: Game days (simulate major outages proactively) │
│ Example: Simulate entire region failure │
│ │
└──────────────────────────────────────────────
```

Common Chaos Experiments

Experiment Target Success Criteria

Pod kill Kubernetes deployment Self-healing, no user impact

Latency injection Service dependency Timeout + circuit breaker works

Database failover Primary → replica Zero downtime failover

Network partition Service A ↔ Service B Graceful degradation

Resource exhaustion CPU/Memory limits Throttling, not crash

Certificate expiry mTLS certs Auto-rotation works

Tools

Tool Use Case Company

Chaos Monkey Kill instances Netflix

Gremlin Commercial platform Gremlin Inc

Litmus Kubernetes native ChaosNative

PowerfulSeal Pod/VMI chaos Bloomberg

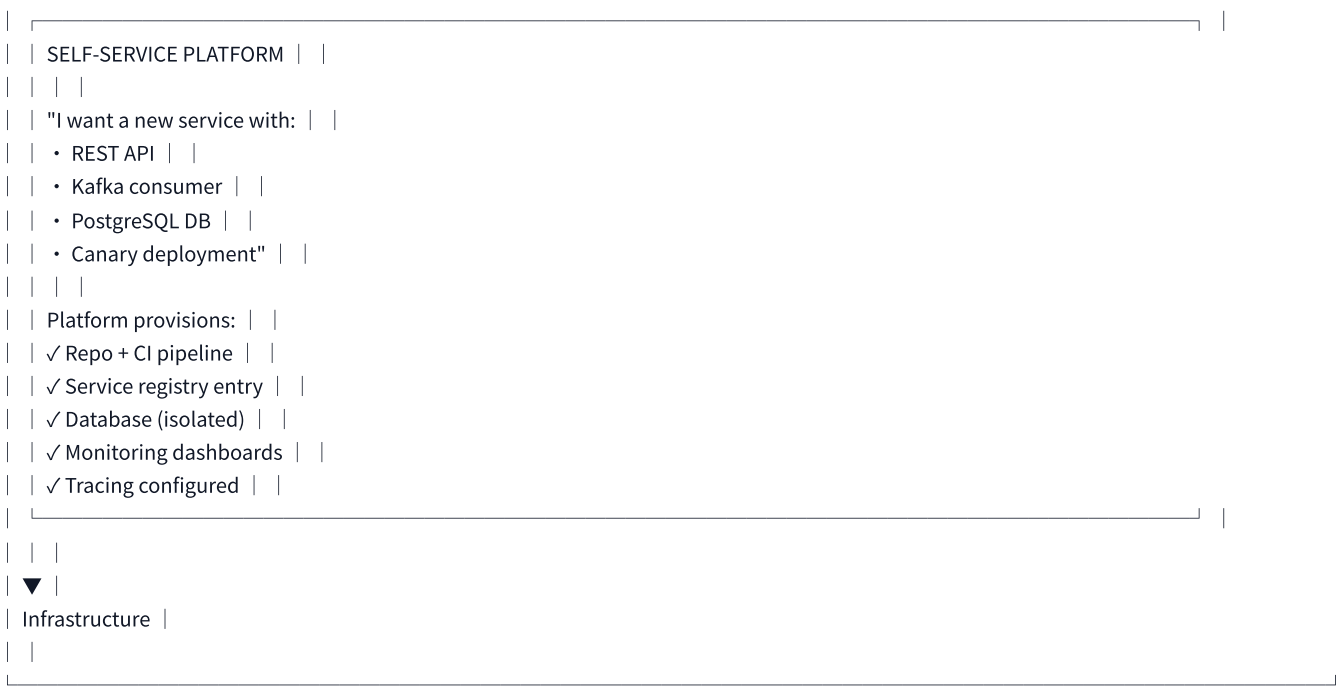
Interview Insight: "Chaos engineering isn't about breaking things randomly. It's about building confidence in system resilience by running controlled experiments."

TOPIC 10: PLATFORM ENGINEERING FOR MICROSERVICES

Internal Developer Platform (IDP) — Golden Paths

text

```
┌──────────────────────────────────────────────┐
│ PLATFORM AS A PRODUCT — GOLDEN PATHS │
├──────────────────────────────────────────────┤
│ │
│ Developer Experience │
│ │ │
│ ▼ │
```



Platform Team Responsibilities

Responsibility	Without Platform	With Platform
Kubernetes clusters	Each team learns k8s	Abstracted away
Service discovery	Complex setup	Built-in
Observability	Self-configure	Auto-instrumented
Secrets management	DIY	Vault integration
CI/CD	Each team builds	Standardized pipelines

When to Build Platform

text

Small team (<20 devs): NO platform → Too much overhead

Growing team (20-100 devs): ONE platform engineer → Start tooling

Mature org (100+ devs): Dedicated platform team → 1:10 ratio

Large org (500+ devs): Platform as internal product → 1:20 ratio

📋 ADVANCED TOPICS QUICK REFERENCE

One-Line Summaries for Interview

Topic	One-Line Summary
2PC	Distributed ACID fails in microservices due to locks and network partitions
TCC	Try-Confirm-Cancel gives short-term reservation before execution
Event Sourcing	Store events, compute state; audit trail is the source of truth
CDC	Listen to DB transaction logs to publish events without code changes
Service Mesh	Offload resilience, observability, security to infrastructure layer
Strangler Fig	Incrementally replace monolith by intercepting and rerouting requests
DORA Metrics	Measure DevOps maturity: frequency, lead time, restore time, failure rate
Cellular Architecture	Deploy complete independent cells for blast radius isolation
Chaos Engineering	Run experiments to build confidence in system resilience
Platform Engineering	Build internal developer platform with golden paths for self-service

CORE PRINCIPLES — Staff Engineer Level

text

1. Cohesion over consistency
   └── Eventual consistency is acceptable; strong coupling is not

2. Autonomy over orchestration
   └── Services should decide their own fate, not be directed centrally

3. Observability over guessing
   └── If you can't measure it, you can't improve it

4. Resilience over correctness
   └── Better to be partially available than fully unavailable
5. Simplicity over completeness
   └── The best architecture is the simplest one that meets requirements
6. Discovery over configuration
   └── Services find each other; don't hardcode anything
7. Idempotency over retry
   └── Without idempotency, retry is dangerous
8. Exponential backoff over immediate retry
   └── Give failing systems time to recover
9. Circuit breaking over endless retry
   └── Know when to stop calling
10. Platform over point solutions
    └── Standardize the infrastructure, not the applications

INTERVIEW POWER PHRASES (Advanced Edition)

"Two-phase commit doesn't work in microservices because network partitions are unavoidable. We use Saga with compensating transactions for eventual consistency."

"Service mesh shifts complexity from application to infrastructure — you pay with operational overhead, but gain standardization across polyglot services."

"The Strangler Pattern is the only safe path to microservices. Anyone proposing a big bang rewrite hasn't experienced one."

"Database per service isn't dogma — it's a trade-off. You pay with distributed transactions and reporting complexity in exchange for independent scalability."

"Chaos engineering is about controlled experiments, not random breaking. We start with 1% blast radius and expand confidence over time."

"Cellular architecture is microservices' answer to blast radius: each cell fails independently, the system as a whole doesn't."

"Platform engineering is a product. Our customers are internal developers. We build golden paths, not golden cages."